Tarefas, alocação e trava (guia para leigo)

PS Adapti 2026.2 · material de estudo

Este PDF cobre **backend + frontend** da Parte 2 (tarefas e trava). A Parte 1 (manutenção) (bugs) está em PDFs separados. Números de linha = código **atual**.

---

0. O que a diretoria pediu (em português claro)

1. Várias **tarefas** por **projeto**.
2. Toda tarefa tem **obrigatoriamente** um **membro**.
3. Um projeto pode ter vários membros (N:N).
4. Um membro **não** pode estar em mais de **2 projetos ativos** ao mesmo tempo.
5. Se tentar o 3º: API **422** + aviso na tela na hora.

Projeto ativo (nossa regra): `data_inicio <= hoje` e `data_fim >= hoje`.

---

1. Glossário

Termo	Significado
N:N / pivot	Muitos membros ↔ muitos projetos. Tabela do meio: `membro_projeto`.
Eloquent	ORM do Laravel (models falam com o banco).
attach / detach	Colocar / tirar membro da pivot.
Service	Classe com regra de negócio (aqui a trava), para não espalhar lógica no controller.
422	Validação falhou; JSON com `message` e `errors`.

Fluxo da trava

```
UI (alocar membro OU criar tarefa)
  → API POST
    → AlocacaoMembroService::alocar
        → já está no projeto? ok
        → projeto inativo? ok (não conta no limite)
        → já tem 2 ativos? → ValidationException 422
        → senão attach na pivot
```

---

2. Modelagem (banco)

Pivot — `database/migrations/2026_10_22_210000_create_membro_projeto_table.php`

Linhas	Conteúdo
11–17	Tabela `membro_projeto` com FKs UUID `membro_id` e `projeto_id`, timestamps, PK composta

Tarefas — `.../2026_10_22_210100_create_tarefas_table.php`

Linhas	Conteúdo
11–19	`id` UUID, `projeto_id`, `membro_id`, `titulo`, `descricao` nullable, `status` default `pendente`

Rodar local (MySQL ligado):

```
cd backend
php artisan migrate
# ou, se preferir recriar tudo:
php artisan migrate:fresh --seed
```

---

3. Models Eloquent

`app/Models/Tarefa.php`

Linhas	O quê
14–24	Constantes de status + lista `STATUSES`
26–32	`$fillable`
34–42	`projeto()` e `membro()` belongsTo

`app/Models/Projeto.php`

Linhas	O quê
31–35	`membros()` belongsToMany via `membro_projeto`
37–40	`tarefas()` hasMany

`app/Models/Membro.php`

Linhas	O quê
36–40	`projetos()` belongsToMany
42–45	`tarefas()` hasMany

---

4. A trava — `app/Services/AlocacaoMembroService.php`

Linhas	O quê
11	`LIMITE_PROJETOS_ATIVOS = 2`
13–19	`projetoEstaAtivo` — compara datas com hoje
21–30	`contarProjetosAtivos` — quantos ativos o membro já tem
38–55	`garantirPodeAlocar` — se ≥ 2 e projeto alvo é ativo e novo → **422** com mensagem em PT

60–67	`alocar` — valida + `attach` se ainda não estiver

Mensagem da linha **52**:

> *Este membro já está alocado em 2 projetos ativos. Remova-o de outro projeto antes de alocar.*

---

5. Controllers e rotas

`TarefaController.php`

Método	Comportamento
`index`	Lista com `projeto` e `membro`
`store`	Valida → `alocar` → cria tarefa (status default `pendente`)
`update`	Se mudou projeto/membro, roda `alocar` de novo
`destroy`	Apaga

`ProjetoMembroController.php`

Rota	Ação
`GET /projetos/{id}/membros`	Lista equipe
`POST /projetos/{id}/membros`	Aloca (`membro_id`) — **passa pela trava**
`DELETE /projetos/{id}/membros/{membro}`	Remove da pivot

`routes/api.php`

Linhas	Rotas
27	`apiResource('/tarefas', ...)`
29–31	rotas de equipe do projeto

`ProjetoController`

- `index` carrega `with('membros')` (contagem na UI).
- `show` carrega `membros` + `tarefas.membro`.

Requests

- `StoreTarefaRequest` / `UpdateTarefaRequest` — projeto, membro, título, status.
- `AttachMembroProjetoRequest` — `membro_id` obrigatório.

Factory / Seeder

- `TarefaFactory.php` — dados fake.
- `TarefaSeeder.php` — aloca até 2 membros por projeto (respeitando a trava) e cria tarefas.
- `DatabaseSeeder` chama `TarefaSeeder` depois de Membros e Projetos.

---

6. Frontend Next.js

Types — `frontend/src/types/tarefa.ts`

Status tipados + `TAREFA_STATUS_LABELS` para a UI.

Actions — `frontend/src/actions/tarefa.ts`

Server actions: CRUD `/tarefas` + `attachMembroProjeto` / `detachMembroProjeto`. Revalidam `/admin/tarefas` e `/admin/projetos`.

Página `/admin/tarefas`

Arquivo	Função
`tarefas/page.tsx`	Header + Suspense
`list-tarefa.tsx`	Busca tarefas+projetos+membros; mobile empilhado / desktop tabela
`form-fields-tarefa.tsx`	Selects nativos (projeto, membro, status) + título
`dialog-create/update/delete-tarefa.tsx`	CRUD com `toastApiError` (mostra a trava)

Equipe do projeto

`projetos/_components/dialog-equipe-projeto.tsx`

- Lista membros alocados + botão remover.
- Select para alocar → `POST .../membros`.
- Se 422 da trava → toast com a mensagem do Laravel.

Ícone de usuários na listagem de projetos (`list-project.tsx` → `DialogEquipeProjeto`).

Sidebar

`sidebar-main.tsx` — link **Tarefas** (`/admin/tarefas`) com ícone `LuListChecks`.

---

7. Como testar a trava (roteiro)

1. MySQL + `php artisan migrate --seed` + `php artisan serve`.
2. Frontend `pnpm dev`.
3. Pegue **um membro** e **3 projetos ativos** (datas cobrindo hoje).
4. Em Projetos → ícone equipe → aloque o membro no projeto A e B → ok.
5. Tente alocar no projeto C → toast: *já está alocado em 2 projetos ativos...*
6. Ou crie tarefa no projeto C com esse membro → mesmo 422.
7. Remova o membro de A → aí consegue alocar em C.

---

8. Frases para apresentação

- "Tarefa sempre com `membro_id` e `projeto_id`; na criação o backend chama `AlocacaoMembroService::alocar`."

- “Projeto ativo = intervalo de datas contém hoje; não inventamos coluna status.”
- “A trava vive no Service (linhas 38–55), não no React — o frontend só exibe o 422.”
- “Pivot `membro_projeto` é o N:N; tarefa é 1:N com projeto e membro.”
- “UI de equipe no projeto é o lugar natural de testar a regra da diretoria.”

---

9. Arquivos novos / alterados (lista fechada)

Backend: migrations pivot e tarefas; `Tarefa` model; relações em `Projeto` / `Membro`; `AlocacaoMembroService`; `TarefaController`; `ProjetoMembroController`; Form Requests; Factory/Seeder; `api.php`; `ProjetoController` (with membros); `DatabaseSeeder`.

Frontend: `types/tarefa.ts`; `types/projeto.ts` (membros?); `actions/tarefa.ts`; pasta `admin/tarefas/*`; `dialog-equipe-projeto.tsx`; `list-project.tsx` (botão equipe); `sidebar-main.tsx`.

Parte 1 (manutenção) frontend permanece no PDF `Correcoes_Frontend_etapa1.pdf`.